

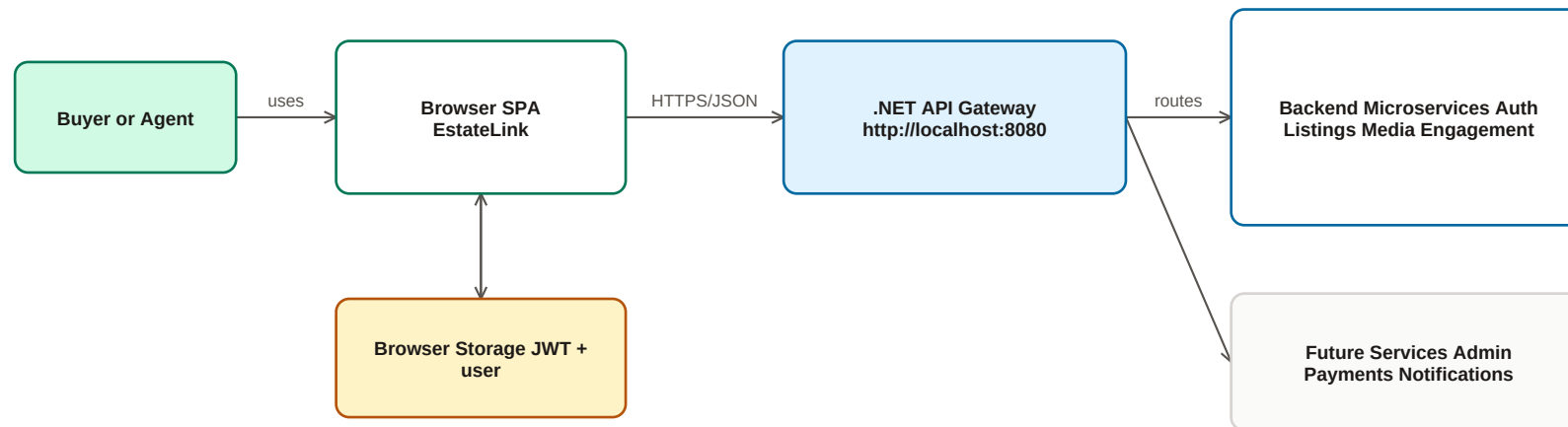
Property Marketplace Frontend

Architecture Design Document

This document describes the implemented React + Vite + TypeScript frontend for EstateLink, a property listing marketplace that connects to an existing .NET API gateway.

Primary runtime: Browser SPA. Backend entry point: `http://localhost:8080` via `VITE_API_BASE_URL`.

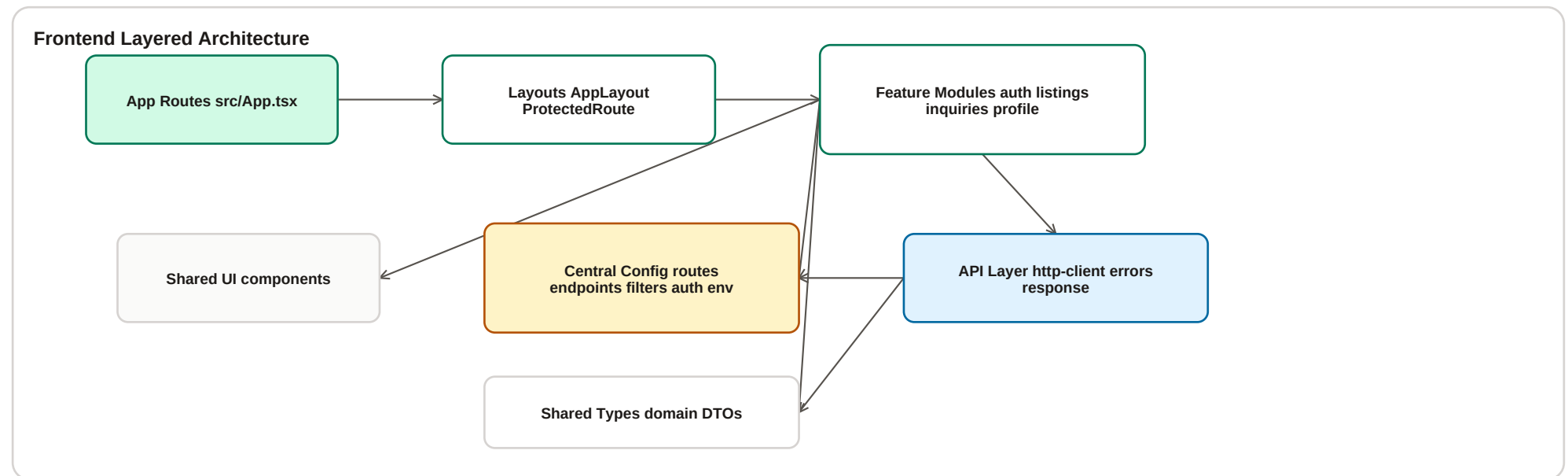
High-Level System Context



1. Executive Summary

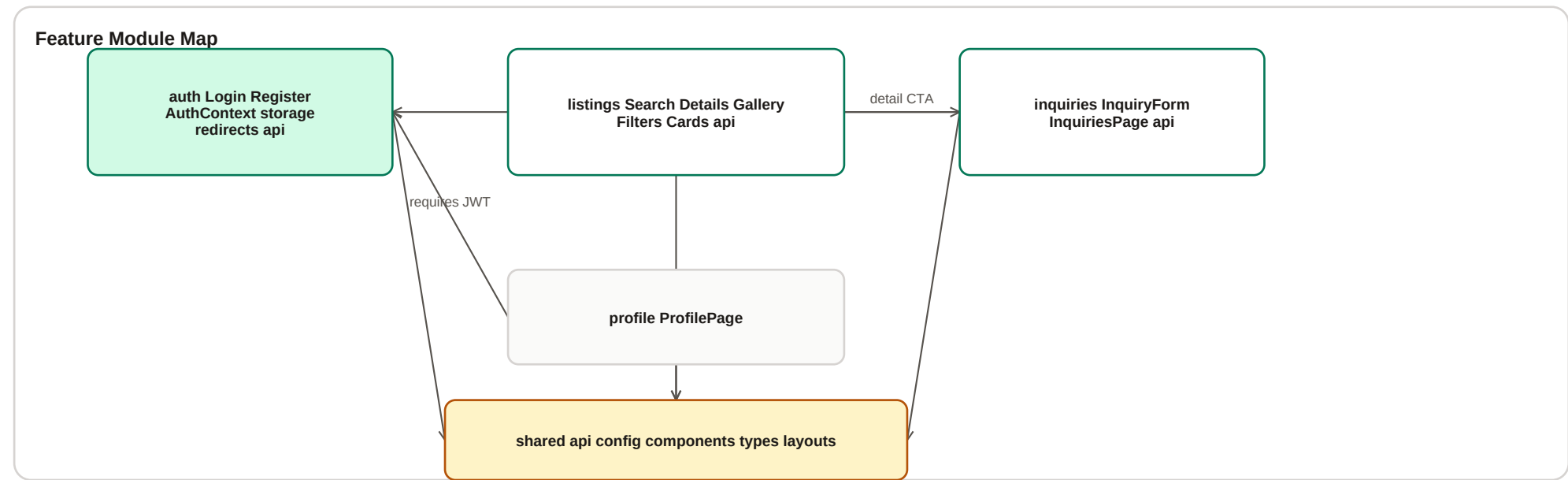
The application is a client-side marketplace frontend with clear separation between configuration, API access, feature modules, reusable UI components, routing, and shared domain types. The architecture is intentionally centralized around `src/config` so junior developers can change routes, endpoint paths, filter defaults, auth keys, navigation, and media fallbacks without hunting through page components.

- Single gateway dependency: all normal calls use `VITE_API_BASE_URL`.
- Feature-owned API wrappers isolate backend response normalization from UI components.
- React Query manages asynchronous loading, error, refresh, and cache behavior.
- JWT handling is contained in auth-storage and consumed by the shared HTTP client.
- Protected routes redirect unauthenticated users to login and route 403 responses to not-authorized.



2. Frontend Architecture

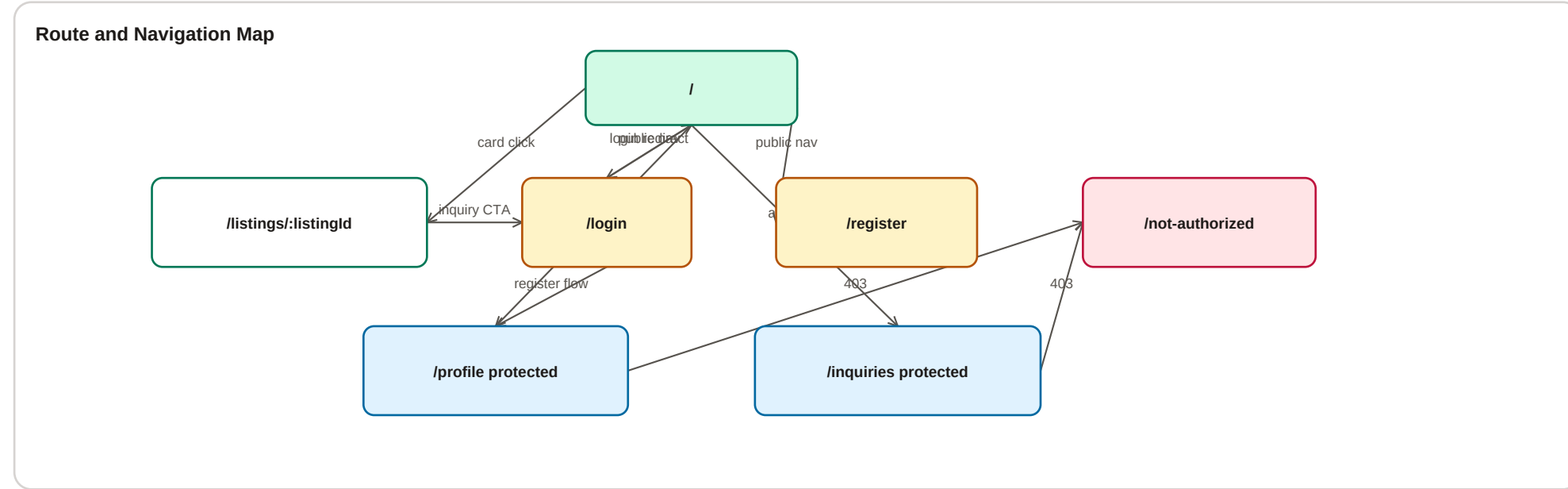
The frontend follows a layered pattern: routing and layouts compose feature pages; feature pages call feature API functions; feature API functions use the shared HTTP client; the HTTP client obtains URLs and auth behavior from central config and auth storage.



Layer	Responsibility	Key Files
Config	Central values for routes, endpoints, filters, auth, navigation, environment, media	src/config/*
API	Fetch wrapper, error extraction, response normalization helpers	src/api/*
Features	Product workflows and feature-specific API functions/components	src/features/*
Components	Reusable UI primitives and shared page states	src/components/*
Layouts	Header, navigation, protected route behavior	src/layouts/*
Types	Shared DTOs and frontend domain types	src/types/domain.ts

3. Route and Navigation Design

Routes are defined in `src/config/routes.ts` and consumed by `App.tsx`, navigation config, protected routes, and feature links. Public navigation exposes Browse, Sign in, and Register. Authenticated navigation exposes Browse, My inquiries, and Profile.

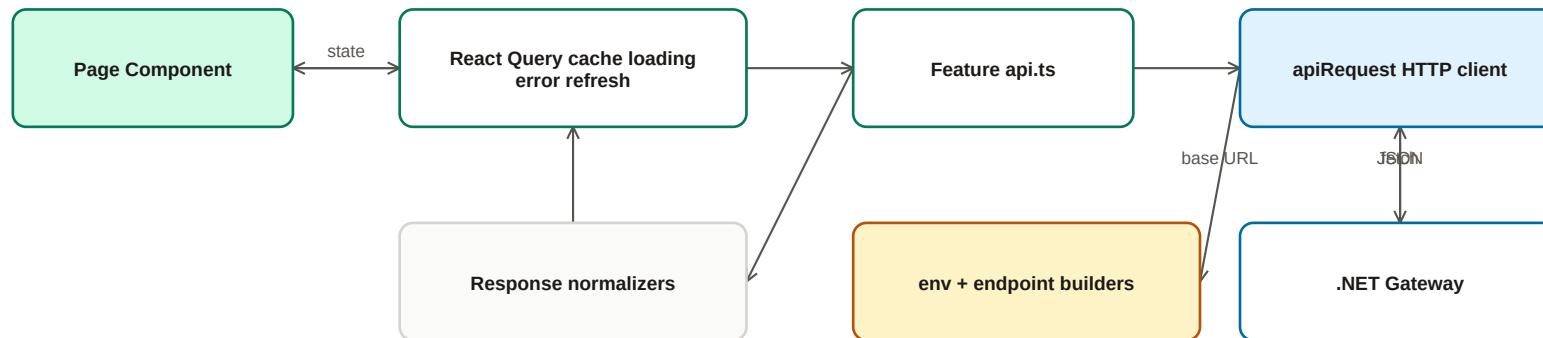


Route	Protection	Purpose
<code>/</code>	Public	Search properties and browse the responsive listing grid
<code>/listings/:listingId</code>	Public with auth-aware CTA	Property details, gallery, facts, location, amenities, agent info
<code>/login</code>	Public	Authenticate and redirect to a sanitized internal path
<code>/register</code>	Public	Register as buyer or agent
<code>/profile</code>	Protected	Show stored user immediately and refresh profile from backend
<code>/inquiries</code>	Protected	List submitted inquiries for the current user
<code>/not-authorized</code>	Public target for 403	Permission error experience

4. API Integration and Data Flow

All backend calls use `apiRequest` from `src/api/http-client.ts`. The HTTP client builds URLs from `env.apiUrl`, attaches bearer tokens for protected calls, serializes JSON bodies, extracts backend error messages, and centralizes 401 and 403 behavior.

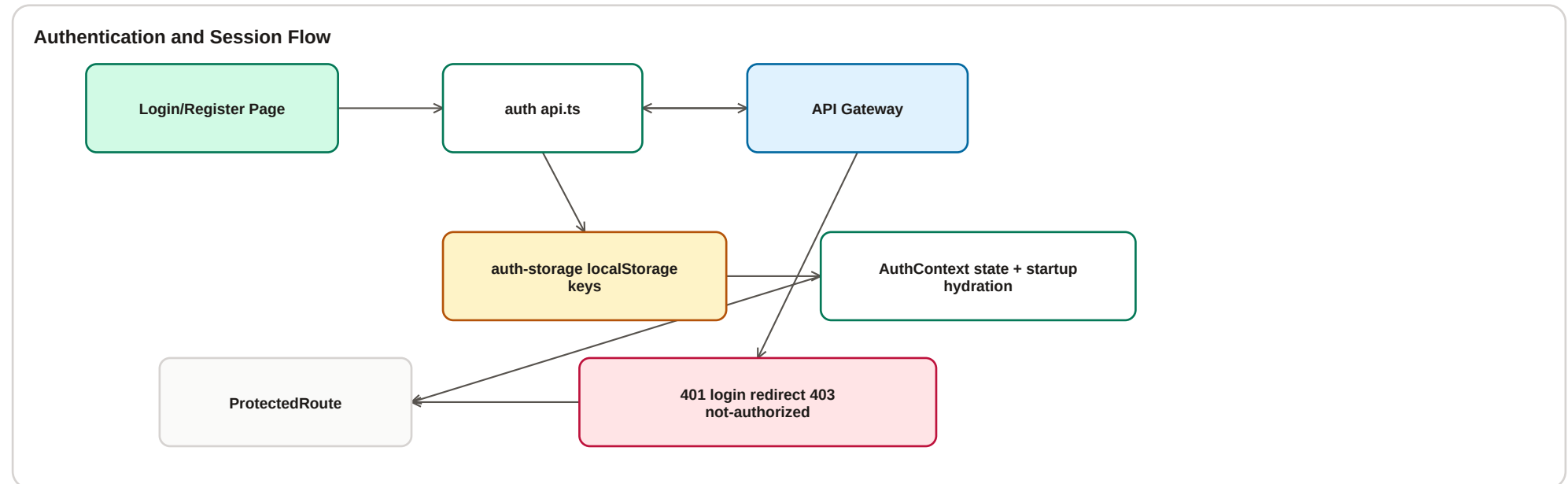
API and UI Data Flow



Feature	Function	Endpoint
Listings	<code>searchListings(filters)</code>	<code>GET /listings</code>
Listings	<code>getListing(listingId)</code>	<code>GET /listings/{listingId}</code>
Listings	<code>getListingMedia(listingId)</code>	<code>GET /media/listings/{listingId}</code>
Auth	<code>login(request)</code>	<code>POST /auth/login</code>
Auth	<code>register(request)</code>	<code>POST /auth/register</code>
Auth	<code>getProfile()</code>	<code>GET /auth/profile</code>
Inquiries	<code>createInquiry(request)</code>	<code>POST /inquiries</code>
Inquiries	<code>getMyInquiries()</code>	<code>GET /inquiries/mine</code>

5. Authentication and Authorization

The backend issues JWT bearer tokens. The frontend stores access and optional refresh tokens through `auth-storage.ts`, hydrates user profile state in `AuthContext`, and attaches Authorization headers in the shared HTTP client. Refresh-token rotation is not implemented because the backend contract is not yet defined for proactive refresh.



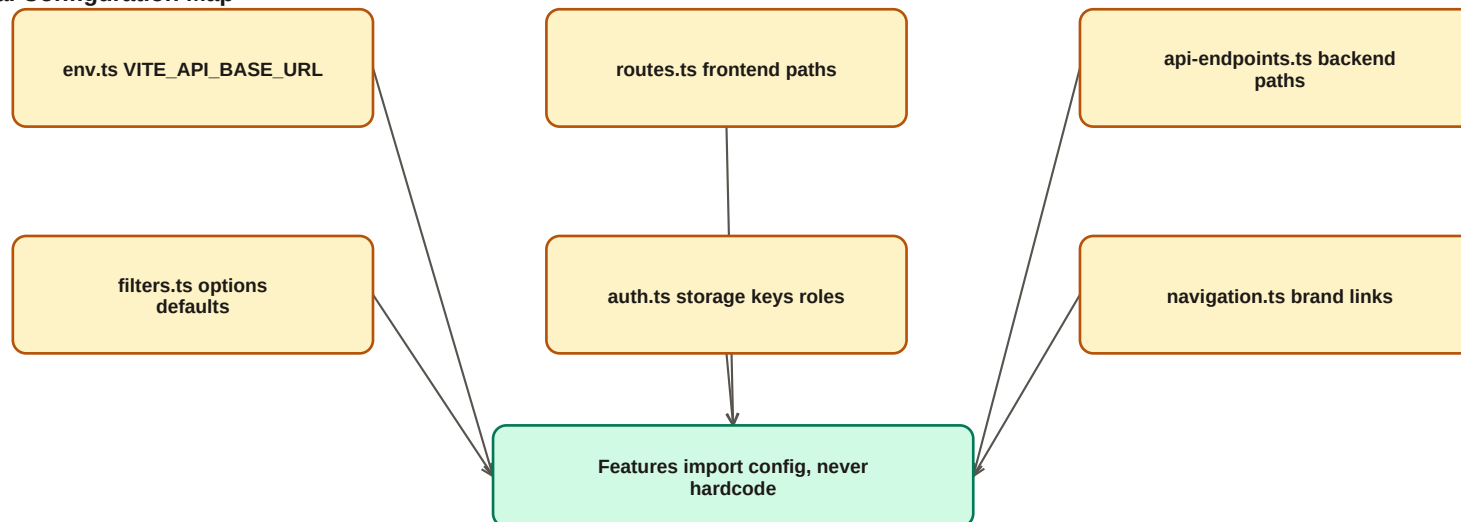
- Login and register accept buyer/agent flows only; admin registration is intentionally not exposed.
- If auth returns a token but no user payload, the frontend stores the token and fetches `GET /auth/profile`.
- On startup, `AuthContext` fetches the profile if a token exists but no user object is stored.
- 401 clears the session and redirects to login with the current internal path as the redirect value.
- 403 redirects protected calls to `/not-authorized`.
- Redirect query values are sanitized to reject external URLs and protocol-relative paths.

6. Listing and Inquiry Workflows

Listing search and listing details are public. Inquiry creation and inquiry history require authentication.

Workflow	Steps
Search	User changes filters -> URL search params update -> React Query calls GET /listings -> results are normalized -> cards render with fallback images when needed
Details	User opens /listings/:listingId -> details and media load independently -> media failure shows a soft warning and placeholders -> details remain usable
Inquiry	Authenticated user submits message -> POST /inquiries -> inquiry query invalidates -> My inquiries can refresh from GET /inquiries/mine
Unauthenticated inquiry	Detail page shows Sign in to inquire -> redirects to /login with sanitized return URL -> after login the user returns to the listing

Central Configuration Map



7. Configuration and Extension Rules

The codebase is designed so common changes start from central configuration. Junior developers should not hardcode new route paths, API endpoint paths, token keys, or listing filter options inside feature pages.

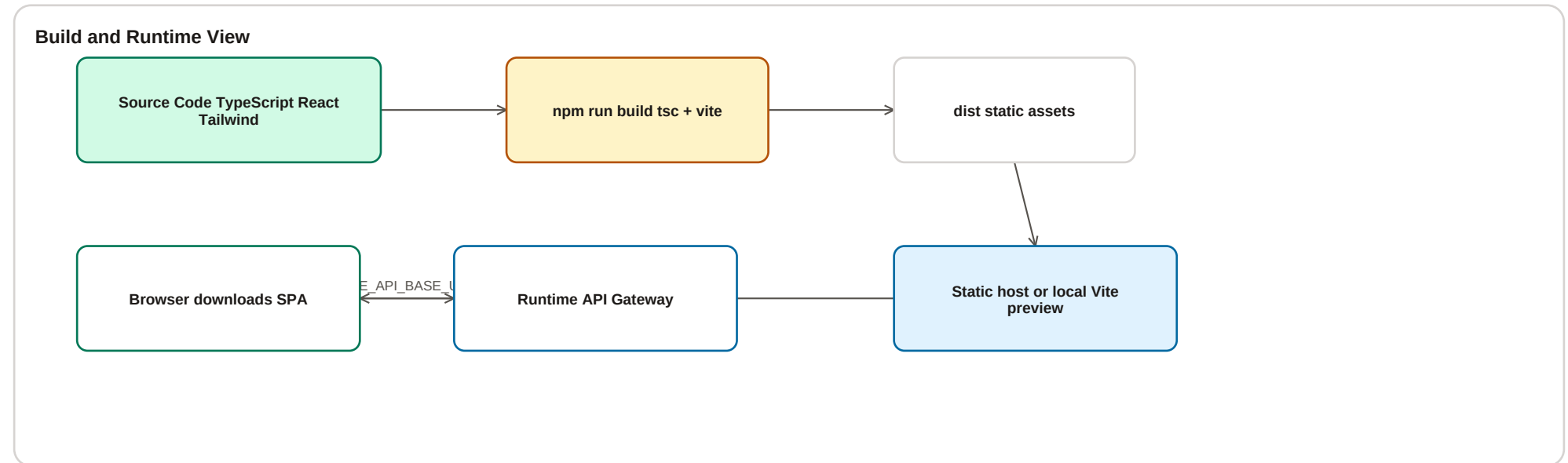
Change Needed	Where To Start	Then Update
New backend call	src/config/api-endpoints.ts	Feature api.ts, domain types, React Query usage
New page	src/config/routes.ts	src/App.tsx, navigation if needed, feature page file
New listing filter	src/config/filters.ts	FilterPanel, ListingSearchFilters type, backend params
New protected page	src/config/routes.ts	ProtectedRoute wrapper and navigation rules
New shared UI	src/components/ui	Use from features; move feature-only components only when shared
Auth storage key change	src/config/auth.ts	No feature page changes expected

Dependency Direction

Allowed: features -> api/config/components/types/layouts. Avoid: shared api/config importing feature page components.

8. Build, Runtime, and Deployment View

The app builds into static assets with Vite. Runtime configuration for the API gateway is read at build/runtime through the Vite environment variable `VITE_API_BASE_URL`. The local dev server runs on `127.0.0.1:3000`.



Command	Purpose
npm install	Install React, Vite, TypeScript, TanStack Query, Tailwind, Lucide, and utility dependencies
npm run dev	Start Vite development server on <code>127.0.0.1:3000</code>
npm run build	Run TypeScript project build and Vite production bundle
npm run preview	Preview the production build locally

9. Security, Reliability, and Quality Notes

- Bearer tokens are read only through auth-storage.ts and attached only by apiRequest.
- External redirect values are rejected to reduce open-redirect risk.
- Backend validation messages are surfaced through the shared error extractor.
- React Query prevents every page from reinventing loading, error, retry, and cache state.
- Response normalizers tolerate common backend shapes such as arrays, items, data, results, and value wrappers.
- Listing media errors do not block the whole property detail page; fallback images keep the page usable.
- Admin, payment, media upload manager, and agent listing management are explicitly outside the current UI scope.

Current Technical Debt and Next Decisions

Topic	Current State	Recommended Next Step
Refresh tokens	Stored if returned, not proactively used	Clarify backend refresh contract before adding rotation
Form validation	Simple client checks plus backend errors	Add schema-based validation if forms grow
Tests	Build verification exists; no automated UI tests yet	Add focused route/API component tests or Playwright smoke tests
Admin and agent workflows	Out of current frontend scope	Implement as separate feature modules behind role checks

10. Appendix: Implemented Source Map

Path	Notes
src/App.tsx	Route registration for public and protected screens
src/config/*	Central app configuration for env, routes, endpoints, filters, auth, media, navigation
src/api/http-client.ts	Shared HTTP client, bearer token attachment, 401/403 behavior
src/features/auth/*	Login, register, auth context, storage, redirect sanitation, auth API
src/features/listings/*	Search, details, gallery, cards, filter URL serialization, listing response normalization
src/features/inquiries/*	Inquiry creation and authenticated inquiry history
src/features/profile/ProfilePage.tsx	Protected profile display with backend refresh
src/components/ui/*	Reusable UI primitives
src/types/domain.ts	Shared frontend DTOs and domain models

Document Generation Notes

This PDF was generated from the implemented codebase and README. It should be regenerated after meaningful architecture changes such as adding admin, agent listing management, payment checkout, or media upload workflows.